

SNT

Learned or Memorized ? Quantifying Memorization Advantage in LLMs

The LLM Revolution

They are use everywhere

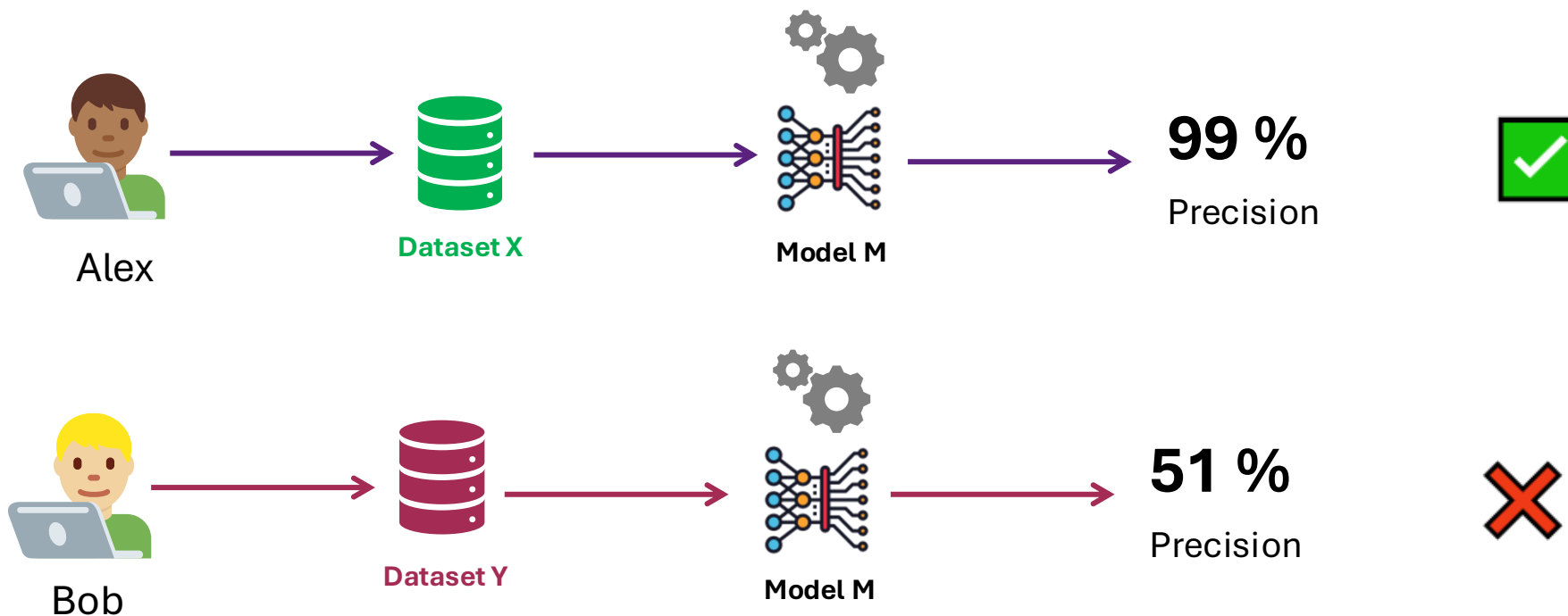


Are they really performant ?



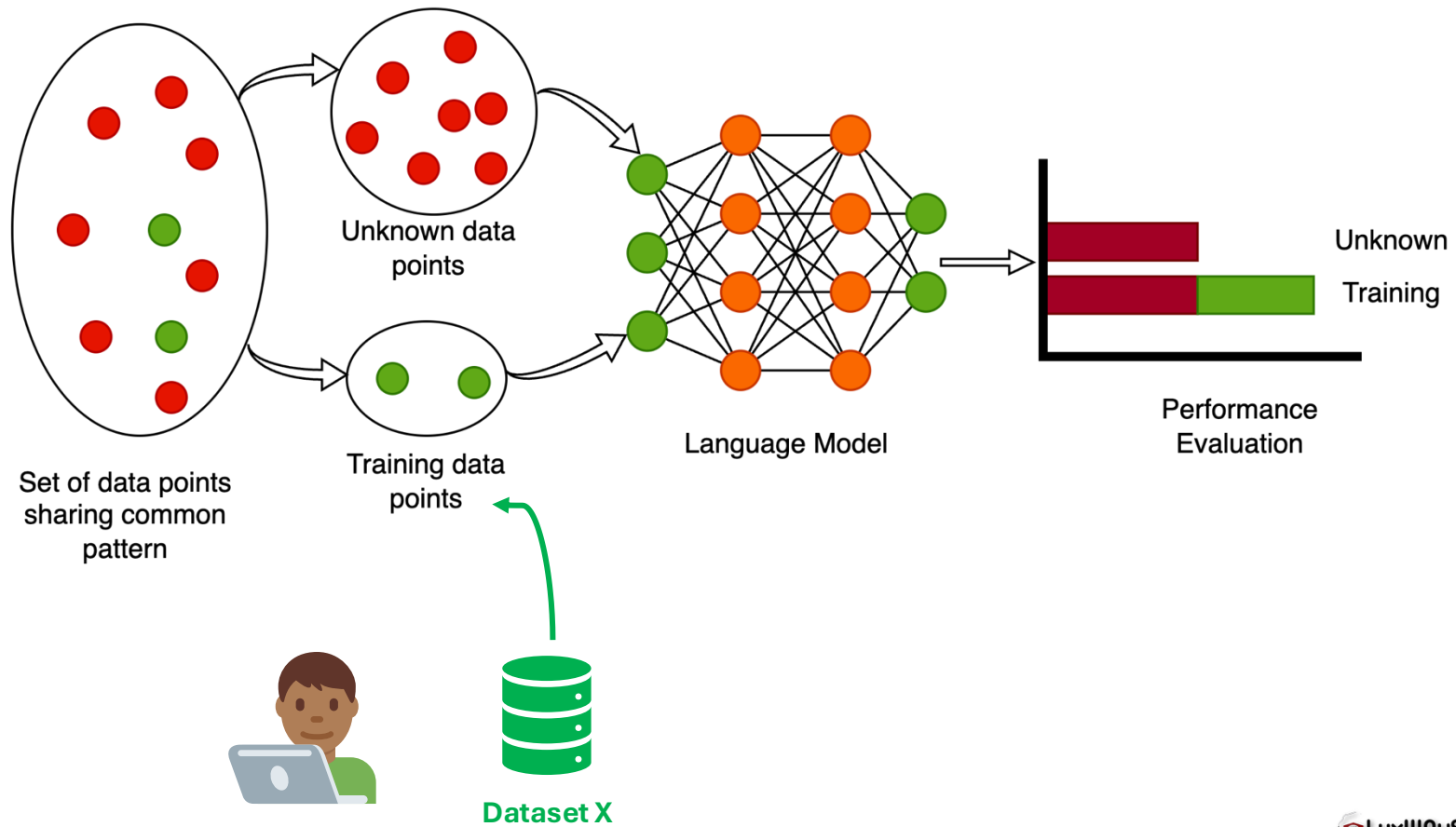
The problem of memorization.

Context



What happened?

Context



What is data memorization ?

Definition.

Data memorization is defined as the lack of generalization of a model that tends to remember and recall details of the data on which it was trained.

Fact Memorization

Refers to the memorization and reproduction of a text or document word for word.

Writing Style Memorization

Allows model to replicate the writing style of a person or document format.

Verbatim

Refers to the memorization and reproduction of a text or document word for word.

Idea Memorization

It allows for the improvement of a model's reasoning capabilities by memorizing the steps to follow to achieve a result.

How to identify LLM's memorization cases

Introduction to Input Perturbation Hypothesis (IPH)

How to detect memorization ?

In or Out training dataset ?

Determine if a datapoint is part of the training dataset

Membership inference attacks against language models via neighbourhood comparison

[PDF] aclanthology.org

[J Mattern](#), [F Mireshghallah](#), [Z Jin](#)... - Findings of the ..., 2023 - aclanthology.org

Abstract Membership Inference attacks (MIAs) aim to predict whether a data sample was present in the training data of a machine learning model or not, and are widely used for ...

☆ Enregistrer Citer Cité 390 fois Autres articles Les 8 versions

Quantifying memorization across neural language models

[PDF] openreview.net

[N Carlini](#), [D Ippolito](#), [M Jagielski](#), [K Lee](#), [F Tramer](#), [C Zhang](#)

The Eleventh International Conference on Learning Representations, 2022 - openreview.net

Large language models (LMs) have been shown to memorize parts of their training data, and when prompted appropriately, they will emit the memorized training data verbatim. This is undesirable because memorization violates privacy (exposing user data), degrades utility (repeated easy-to-memorize text is often low quality), and hurts fairness (some texts are memorized over others). We describe three log-linear relationships that quantify the degree to which LMs emit memorized training data. Memorization significantly grows as

AFFICHER PLUS

☆ Enregistrer Citer Cité 1494 fois Autres articles Les 7 versions

Detecting pretraining data from large language models

[PDF] iclr.cc

[W Shi](#), [A Ajith](#), [M Xia](#), [Y Huang](#), [D Liu](#), [T Blevins](#), [D Chen](#), [L Zettlemoyer](#)

International Conference on Learning Representations, 2024 - proceedings.iclr.cc

Abstract

Although large language models (LLMs) are widely deployed, the data used to train them is rarely disclosed. Given the incredible scale of this data, up to trillions of tokens, it is all but certain that it includes potentially problematic text such as copyrighted materials, personally identifiable information, and test data for widely reported reference benchmarks. However, we currently have no way to know which data of these types is included or in what proportions. In this paper, we study the pretraining data detection

AFFICHER PLUS

☆ Enregistrer Citer Cité 709 fois Autres articles Les 7 versions

How to detect memorization ?

Data Extraction : Sign of private data memorization

Extraction of existing sequences from LLM's.

Analyzing leakage of personally identifiable information in language models

[PDF] arxiv.org

[N Lukas](#), [A Salem](#), [R Sim](#), [S Tople](#), [L Wutschitz](#), [S Zanella-Béguelin](#)

2023 IEEE Symposium on Security and Privacy (SP), 2023 · [ieeexplore.ieee.org](#)

Language Models (LMs) have been shown to leak information about training data through sentence-level membership inference and reconstruction attacks. Understanding the risk of LMs leaking Personally Identifiable Information (PII) has received less attention, which can be attributed to the false assumption that dataset curation techniques such as scrubbing are sufficient to prevent PII leakage. Scrubbing techniques reduce but do not prevent the risk of PII leakage: in practice scrubbing is imperfect and must balance the

AFFICHER PLUS ▾

☆ Enregistrer Citer Cité 513 fois Autres articles Les 5 versions

Extracting training data from large language models

[PDF] usenix.org

[N Carlini](#), [F Tramer](#), [E Wallace](#), [M Jagielski](#)... - 30th USENIX security ..., 2021 - [usenix.org](#)

... that large language **models** memorize and leak individual **training** examples. In particular, ... method for **extracting** verbatim sequences from a **language model**'s **training set** using only ...

☆ Enregistrer Citer Cité 4334 fois Autres articles Les 10 versions

Rethinking llm memorization through the lens of adversarial compression

[PDF] neurips.cc

[A Schwarzschild](#), [Z Feng](#), [P Maini](#)... - Advances in Neural ..., 2024 - [proceedings.neurips.cc](#)

... We examine several unlearning methods using **ACR** to show that they do not substantially ... content are still able to reproduce them with a high **ACR**—in fact, not much smaller than with ...

☆ Enregistrer Citer Cité 129 fois Autres articles Les 6 versions

Measuring memorization in language models via probabilistic extraction

[PDF] aclanthology.org

[J Hayes](#), [M Swanberg](#), [H Chaudhari](#), [I Yona](#), [I Shumailov](#), [M Nasr](#), [CA Choquette-Choo](#)...

Proceedings of the 2025 Conference of the Nations of the Americas ..., 2025 · [aclanthology.org](#)

Abstract

Large language models (LLMs) are susceptible to memorizing training data, raising concerns about the potential extraction of sensitive information at generation time. Discoverable extraction is the most common method for measuring this issue: split a training example into a prefix and suffix, then prompt the LLM with the prefix, and deem the example extractable if the LLM generates the matching suffix using greedy sampling. This definition yields a yes-or-no determination of whether extraction was successful with

AFFICHER PLUS ▾

☆ Enregistrer Citer Cité 50 fois Autres articles Les 7 versions

How to detect memorization ?

Evaluation of the output

Modelisation of the landscape of LLM's output features given an input.

Generalization or memorization: Data contamination and trustworthy evaluation for large language models

[\[PDF\] aclanthology.org](#)

[Y Dong](#), [X Jiang](#), [H Liu](#), [Z Jin](#), [B Gu](#), [M Yang](#), [G Li](#)

Findings of the Association for Computational Linguistics: ACL 2024, 2024 · [aclanthology.org](#)

Abstract

Recent statements about the impressive capabilities of large language models (LLMs) are usually supported by evaluating on open-access benchmarks. Considering the vast size and wide-ranging sources of LLMs' training data, it could explicitly or implicitly include test data, leading to LLMs being more susceptible to data contamination. However, due to the opacity of training data, the black-box access of models, and the rapid growth of synthetic training data, detecting and mitigating data contamination for LLMs faces significant

AFFICHER PLUS ▾

☆ Enregistrer Citer Cité 285 fois Autres articles Les 5 versions »

From Memorization to Reasoning in the Spectrum of Loss Curvature

[\[PDF\] arxiv.org](#)

[J Merullo](#), [S Vatsavaya](#), [L Bushnaq](#), [O Lewis](#)

arXiv preprint arXiv:2510.24256, 2025 · [arxiv.org](#)

We characterize how memorization is represented in transformer models and show that it can be disentangled in the weights of both language models (LMs) and vision transformers (ViTs) using a decomposition based on the loss landscape curvature. This insight is based on prior theoretical and empirical work showing that the curvature for memorized training points is much sharper than non memorized, meaning ordering weight components from high to low curvature can reveal a distinction without explicit labels. This

AFFICHER PLUS ▾

☆ Enregistrer Citer Cité 1 fois Autres articles Les 2 versions »

[PDF] Forget What You Know about LLMs Evaluations--LLMs are Like a Chameleon

[\[PDF\] aclanthology.org](#)

[N Cohen-Inger](#), [Y Elisha](#), [B Shapira...](#) · arXiv preprint arXiv ..., 2025 · [aclanthology.org](#)

Large language models (LLMs) often appear to excel on public benchmarks, but these high scores may mask an overreliance on datasetspecific surface cues rather than true language understanding. We introduce the Chameleon Benchmark Overfit Detector (C-BOD), a meta-evaluation framework designed to reveal such overfitting. C-BOD systematically rephrases benchmark inputs via a parameterized transformation that preserves semantic content and labels, enabling the detection of performance degradation indicative of superficial pattern ...

☆ Enregistrer Citer Cité 15 fois Autres articles Les 5 versions »

Memorization vs Generalization ?

Is my model smart or have a good memory ?

Input	Output
Memorization	
Complete the following text: Hagrid raised a gigantic fist and knocked three times on the castle door.	Hagrid raised a gigantic fist and knocked three times on the castle door. The sound echoed through the ancient stone walls, reverberating like a drumbeat in the silent night.
Complete the following text: Hagrid graised a giganticfist and knocked [] times on the door.	Hagrid graised a giganticfist and knocked [] times on the door. Step 1: Corrected the spelling errors in the given text. Step 2: Added a number to complete the sentence.
Generalization	
Complete the following text: Albert Einstein was a theoretical physicist who is known for developing the theory of relativity,	Albert Einstein was a theoretical physicist who is known for developing the theory of relativity, which revolutionized our understanding of space, time, and gravity.
Complete the following text: AlbertEinstein was a theoretical [] who is known for developing the theory of relative .	AlbertEinstein was a theoretical [] who is known for developing the theory of relative. This groundbreaking theory, which includes both the Special Theory of Relativity and the General Theory of Relativity, revolutionized our understanding of space, time, and gravity.

Prompt
Write a python function to find the sum of fourth power of n natural numbers.
Answer
def func(n): return sum([i**4 for i in range(1,n+1)])



Prompt
Write a python function to find the sum of fourth power of n natural numbers.
Answer
def sum_of_fourth_powers(n): return sum([i**4 for i in range(1, n+1)])



Perturbed Prompt
Create a Python function that calculates the sum of the first 'n' natural numbers, each raised to the power of four.
Answer
def sum_fourth(n): return 4 if n == 1 else pow(n, 4) + sum_fourth(n - 1)



Perturbed Prompt
Create a Python function that calculates the sum of the first 'n' natural numbers, each raised to the power of four.
Answer
def calculate_sum(n): return sum([i**4 for i in range(1, n+1)])



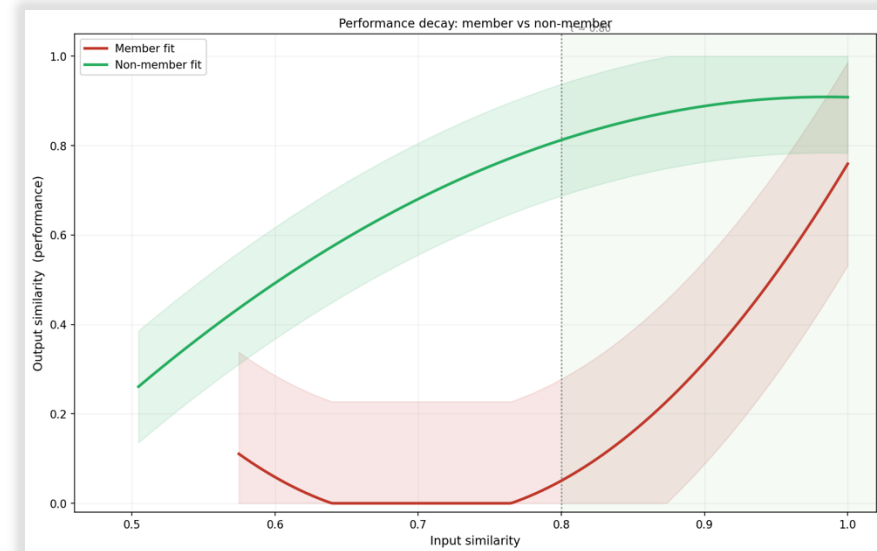
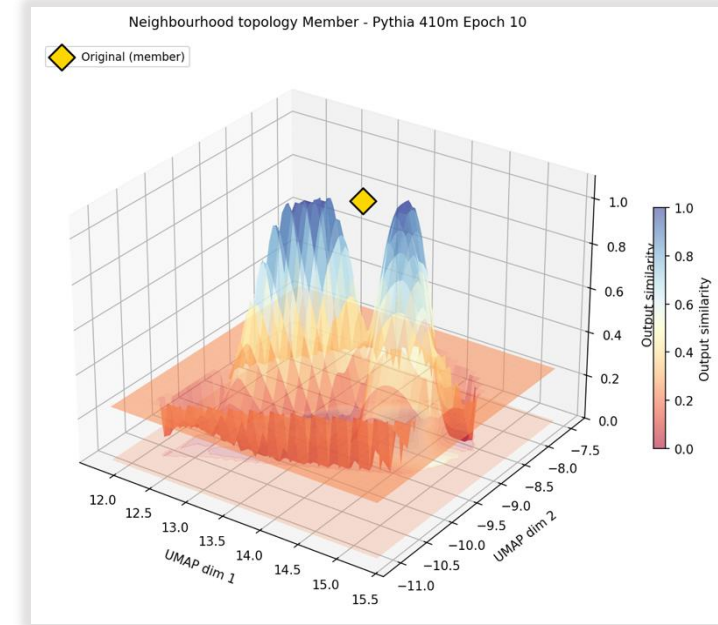
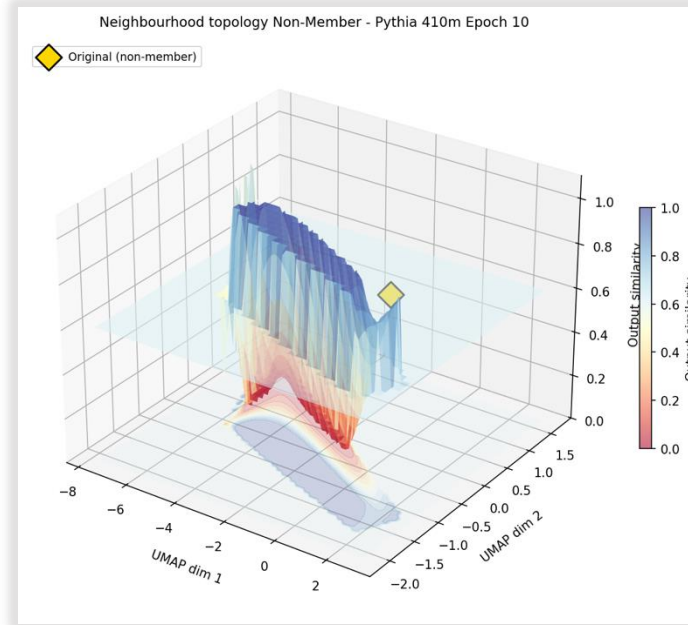
StarCoder 2 - 15B

CodeLlama - 13B

Memorization vs Generalization ?

Is my model smart or have a good memory ?

A small perturbation applied on memorized instance cause an important performance decrease.



Memorization vs Generalization ?

Is my model smart or have a good memory ?

MEMORIZATION

- ✗ **Narrow attractor**
Specific training instance occupies a tiny region of input space
- ✗ **Brittle**
Minor input changes cause sharp, immediate performance degradation
- ✗ **Recall, not reason**
High score reflects exact match, not learned principle
- ✗ **Sensitive to noise**
Even semantics-preserving perturbations disrupt output quality

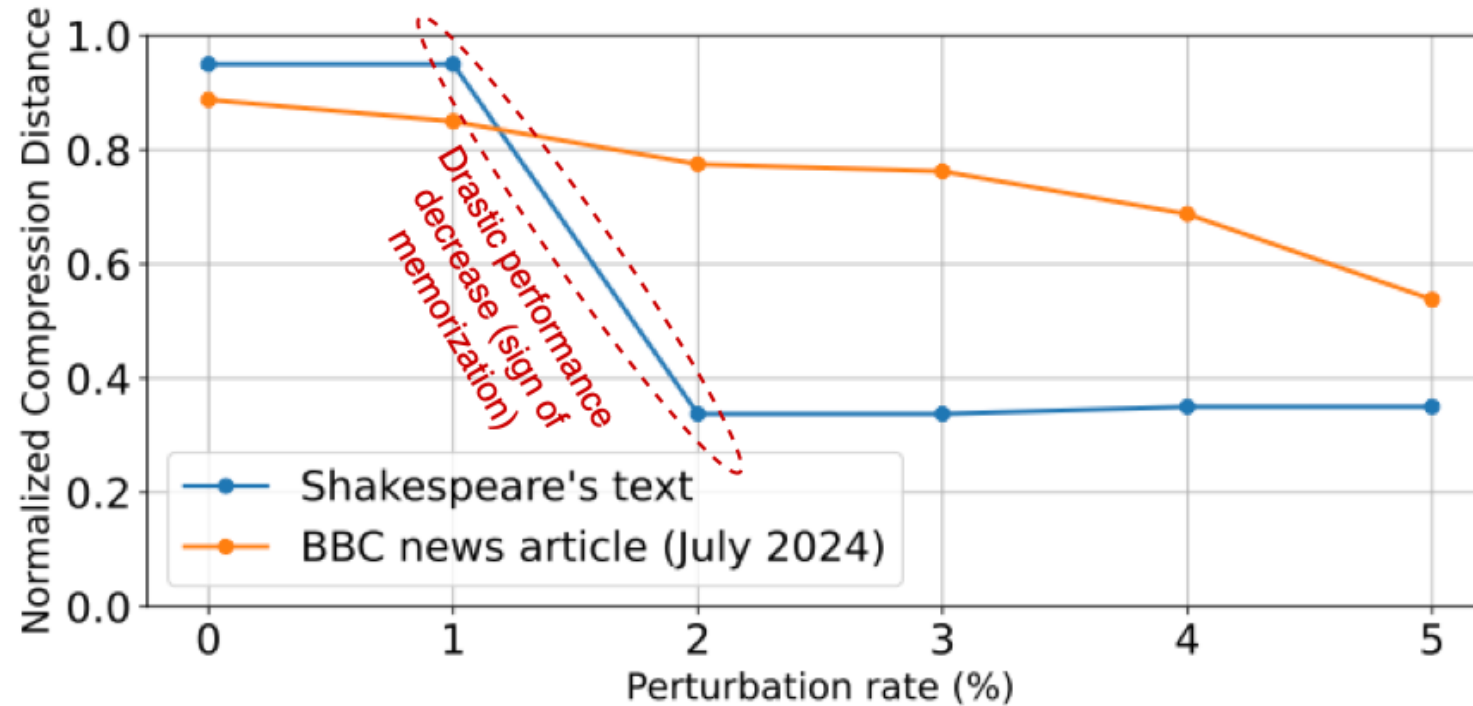
GENERALIZATION

- ✓ **Broad basin**
Learned pattern is stable across a large neighborhood of inputs
- ✓ **Robust**
Performance remains consistent under semantics-preserving changes
- ✓ **Principle learned**
Model extracts transferable patterns from training distribution
- ✓ **Stable under noise**
Perturbations leave output essentially unchanged

Or they just have a good memory.

The Input Perturbation Hypothesis.

Model that have **memorized** content exhibit **high sensitivity**
to input perturbation

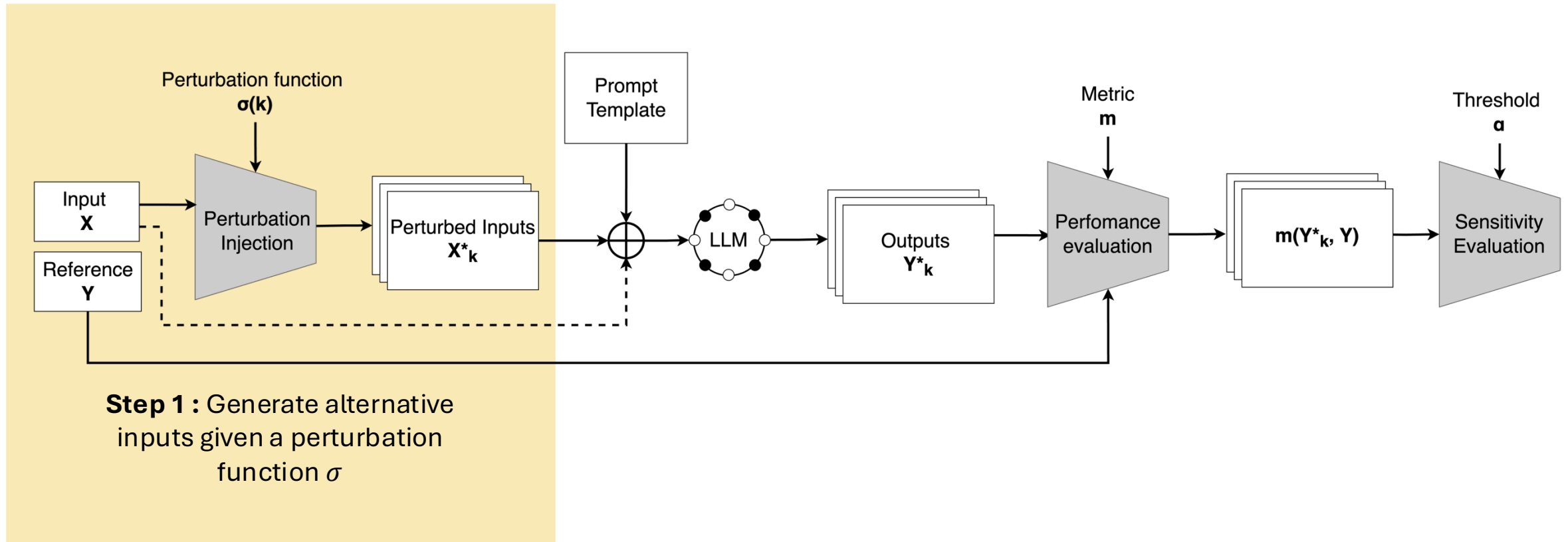


Toward the operationnnalization of IPH

Introduction to **P**erturbation **A**nalysis for **R**evealing Language model
memorization (**PEARL**)

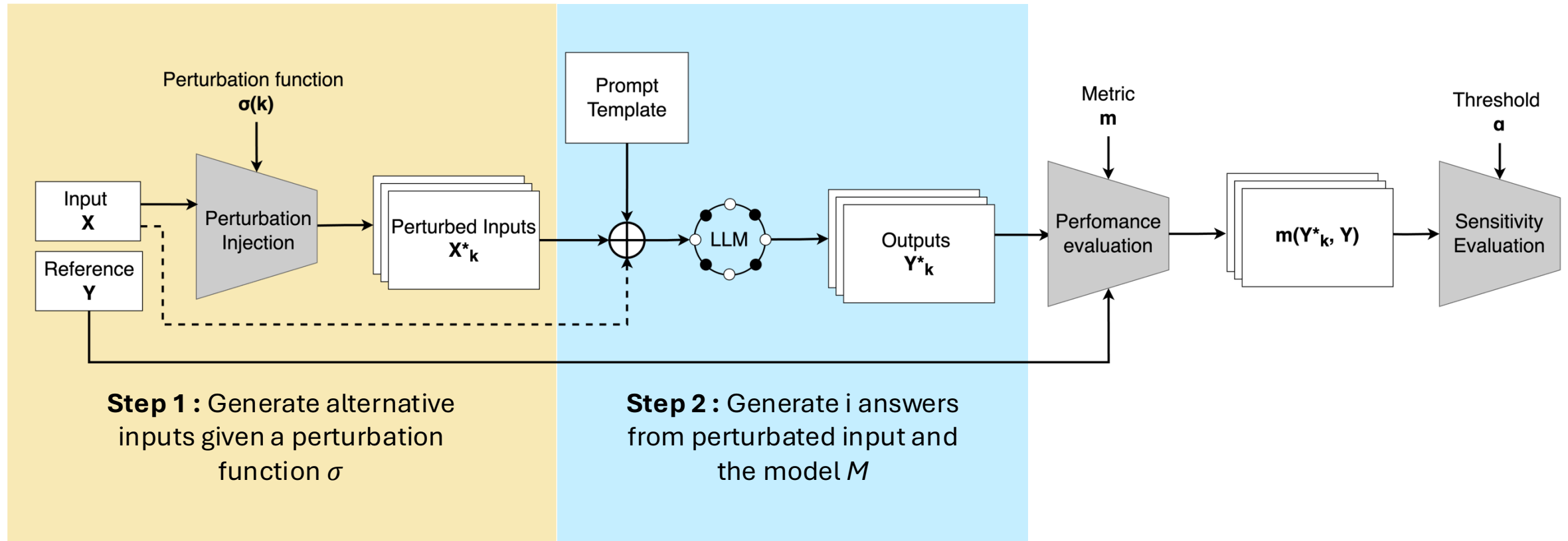
Application of IPH.

Introduction to PEARL Framework.



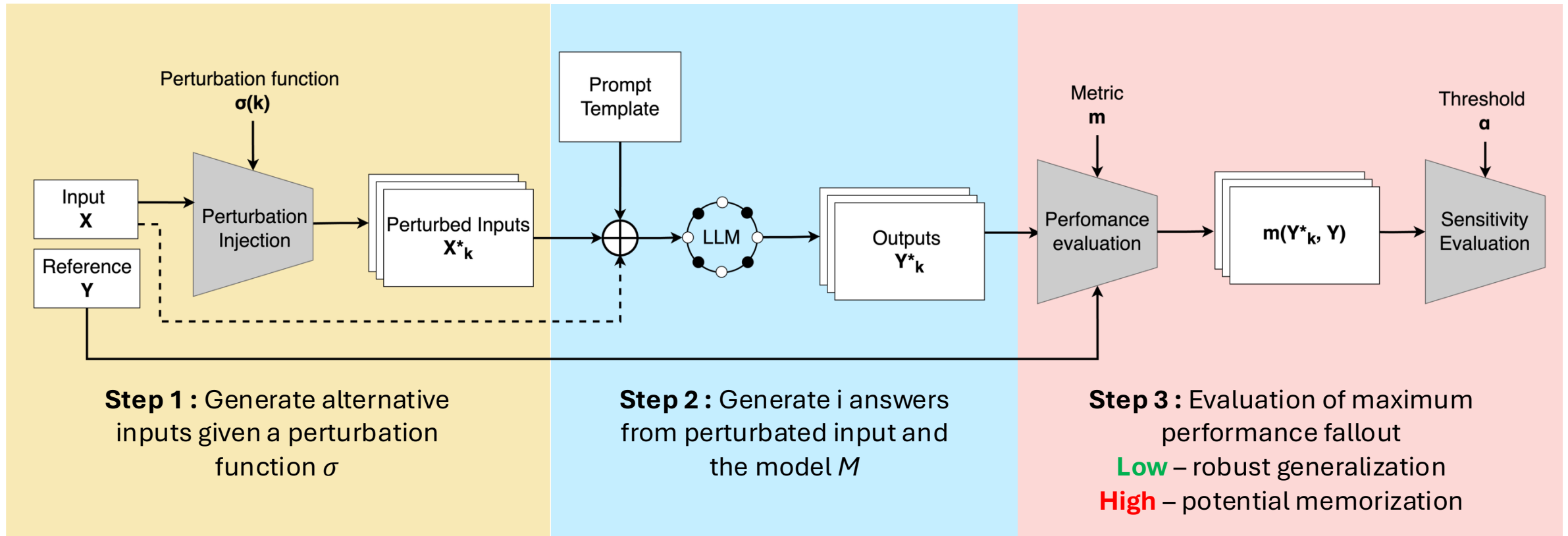
Application of IPH.

Introduction to PEARL Framework.



Application of IPH.

Introduction to PEARL Framework.



Evaluation of PEARL.

Experiment Setup

TARGETED MODEL : **PYTHIA SUITE**



PILE



REFINE-WEB

Table 1: Transformation operators used to construct $N(x)$.

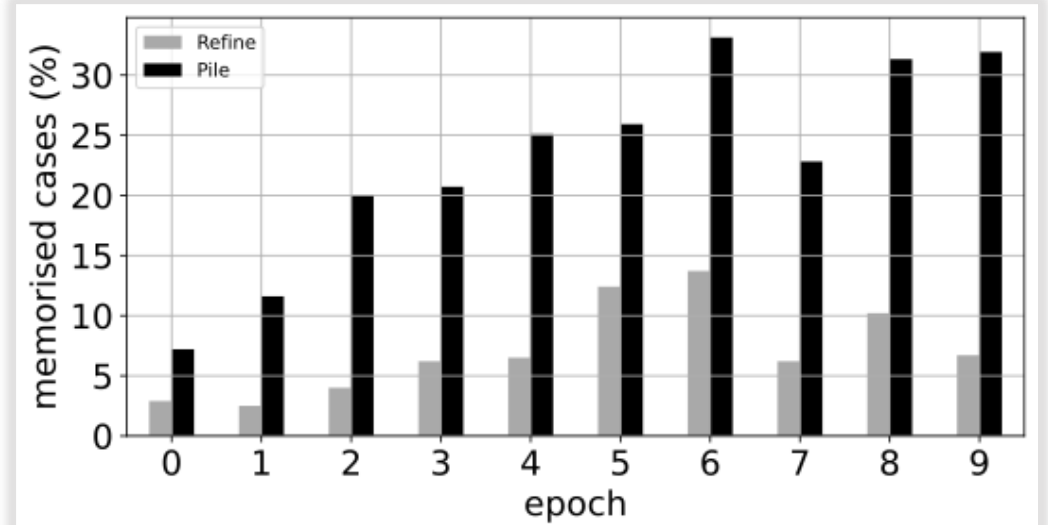
Family	Operator	Description
Surface	whitespace	Insert or remove whitespace characters
	change_case	Mutate character casing at random positions
	swap_chars	Swap adjacent characters at random positions
Advanced	synonyms	Replace tokens with synonyms
	bitflip	Apply bit-level character corruption

Evaluation of PEARL.

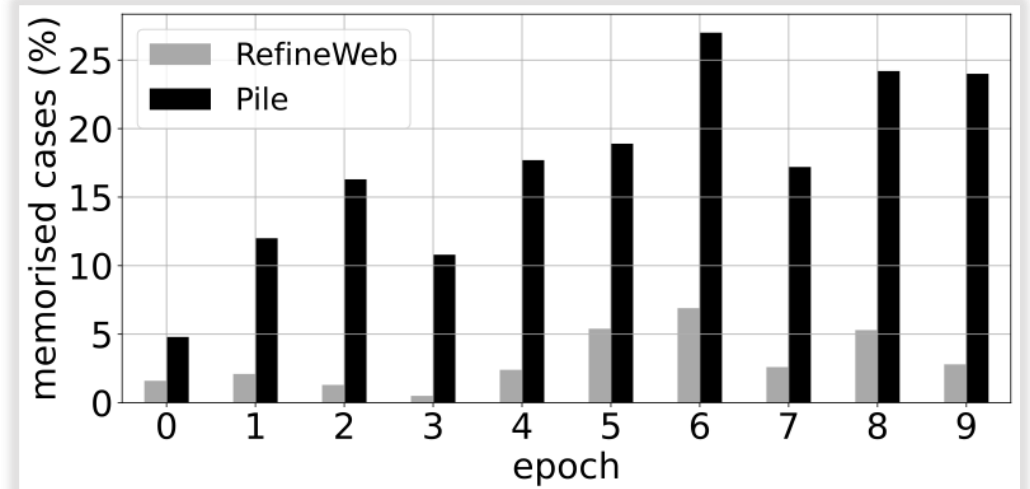
Memorized instances detection quality.

Table 2: Detection performance of PEARL and baselines on Pythia-1.4B (1000 members / 1000 non-members from Mimir). AUC is the best value across 10 epochs. $\#\hat{M}_E / \#\hat{M}_G$ = instances flagged as memorized in the member / non-member split at the best epoch.

Method	Access	Best AUC	$\#\hat{M}_E$	$\#\hat{M}_G$
Logit access required				
Loss Yeom et al. (2018)	Gray-box	0.686	516	226
Min-K% Shi et al. (2023)	Gray-box	0.685	525	248
Neighborhood Attack Mattern et al. (2023)	Gray-box	0.687	585	284
ACR Schwarzschild et al. (2024)	Gray-box	0.586	320	142
Require prior knowledge of training data				
N-Gram ($n = 5$) Carlini et al. (2021a)	Black-box	0.970	889	23
N-Gram ($n = 7$) Carlini et al. (2021a)	Black-box	0.968	504	93
N-Gram ($n = 13$) Carlini et al. (2021a)	Black-box	0.957	360	35
No access to training data / logits				
CDD Dong et al. (2024)	Black-box	0.565	188	126
PEARL (A_{mean})	Black-box	0.777	592	144



Transformation applied : Bilflip

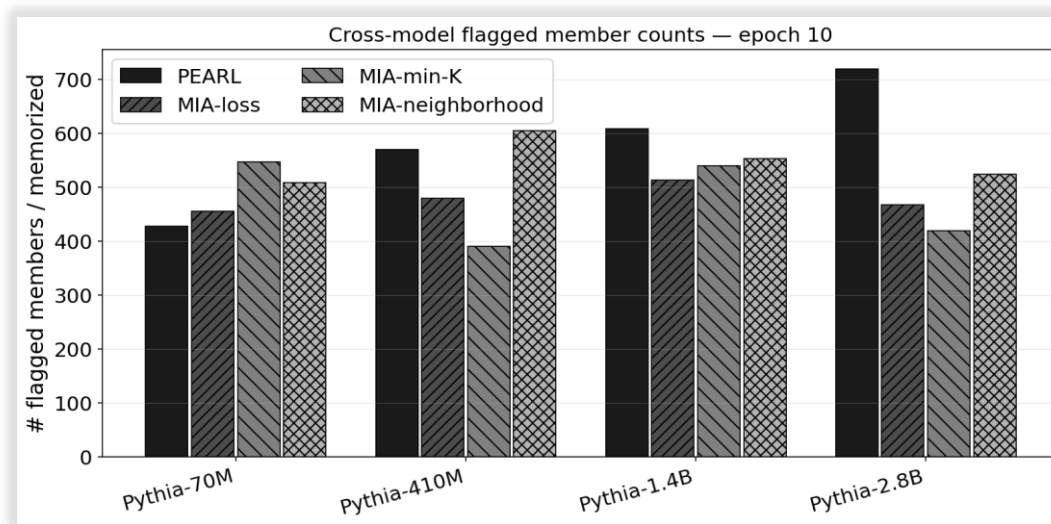
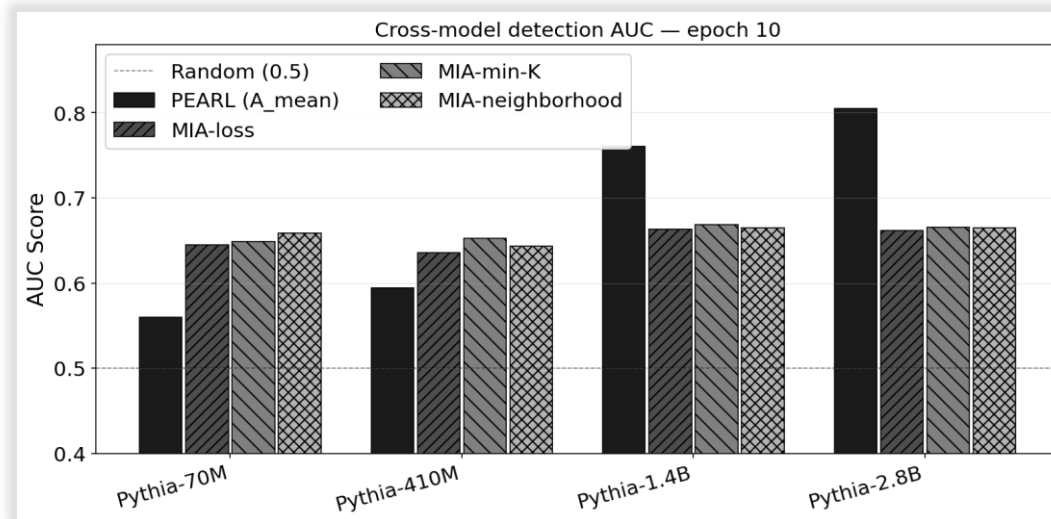


Transformation applied : Synonym substitution

Evaluation of PEARL.

Impact of model size

Larger models are more sensible to input perturbation against memorized instances



Evaluation of PEARL.

Impact of perturbed input samples size

- Sample the perturbed inputs following the cosine similarity with the original.
- Considered 5 similarity buckets : **$[1; 0.9[$ – $[0.9; 0.8[$ – $[0.8; 0.7[$ – $[0.7; 0.6[$ – $[0.6; 0.5[$**
- Considered for each cummulative buckets (**$[1; X]$**), select k samples

Table 7: AUC as a function of the number of perturbations per sample K on Pythia-1.4B at epoch 10 (A_{mean} , 30 random trials). *Targeted*: only $[0.90, 1.00]$ bucket. *Pooled*: all five buckets.

K	Targeted ($\alpha \geq 0.90$)		Pooled ($\alpha \geq 0.50$)	
	AUC (mean)	AUC (std)	AUC (mean)	AUC (std)
1	0.759	0.019	0.677	0.036
2	0.794	0.011	—	—
3	0.814	0.007	0.728	0.030
4	0.826	0.004	—	—
5	0.836	0.000	0.749	0.025
10	—	—	0.760	0.012
15	—	—	0.761	0.006
25	—	—	0.761	0.000

Prior results : what to retain.



Memorization instances are more sensitive to input perturbation:

Thus, it is possible to identify them by comparing the performance variation under perturbation.



PEARL overcome MIA approaches to detect memorization with large models:

No need to get access to logits or internal model parameters.



Larger models memorize more :

PEARL's memorized pattern is more important with large models.

IPH in Software Engineering ?

Is my LLMs smart in SE task or have a good memory.

LLMs achieve SOTA on code generation, vulnerability detection, test generation, and program repair



Static benchmark leaked : Abnormally increase of the model performance during the evaluation

Exploration of static benchmarks memorization?

5 SE Tasks considered

Code Generation	5 benchmarks	Pass@1	HumanEval	APPS	MBPP	LBPP	XLCost	
Test Generation	3 benchmarks	Coverage	BigCodeBench	TestEval	QuixBugs			
Program Repair	3 benchmarks	Pass@1	Defects4J	QuixBugs	Condefects			
Vulnerability Detection	6 benchmarks	F1	CVEFixes	Design	ReVeal	DiverseVul	VulDetectBench	VulnPatchPair
Code Summarization	2 benchmarks	BLEU	CodeSearchNet (Python)	TLCodeSum				

Exploration of static benchmarks memorization?

8 models considered

Model	Family	Params	Key characteristic
DeepSeek-Coder-V2	DeepSeek	16B	60% code, 10% math, 30% NL
Qwen2.5-Coder	Qwen	14B	Hybrid local/global attention
StarCoder2	BigCode	15B	Stack V2 + APPS/CodeContest
CodeLlama	Meta/Llama2	13B	Cascade finne-tuning
Codestral	Mistral	22B	80+ programming languages
OpenCoder	RefineCode	8B	960B tokens, 607 languages
WizardCoder	StarCoder	33B	Evol-Instruct self-instruction
Magocoder	CodeLlama-Py	7B	OSS-Instruct methodology

Toward PEARL's Application?

Code Perturbation

ORIGINAL

```
def is_valid_parenthesization(parens):
    depth = 0
    for paren in parens:
        if paren == '(':
            depth += 1
        else:
            depth -= 1
            if depth < 0:
                return False
    return True
```

LIGHT PERTURBATION

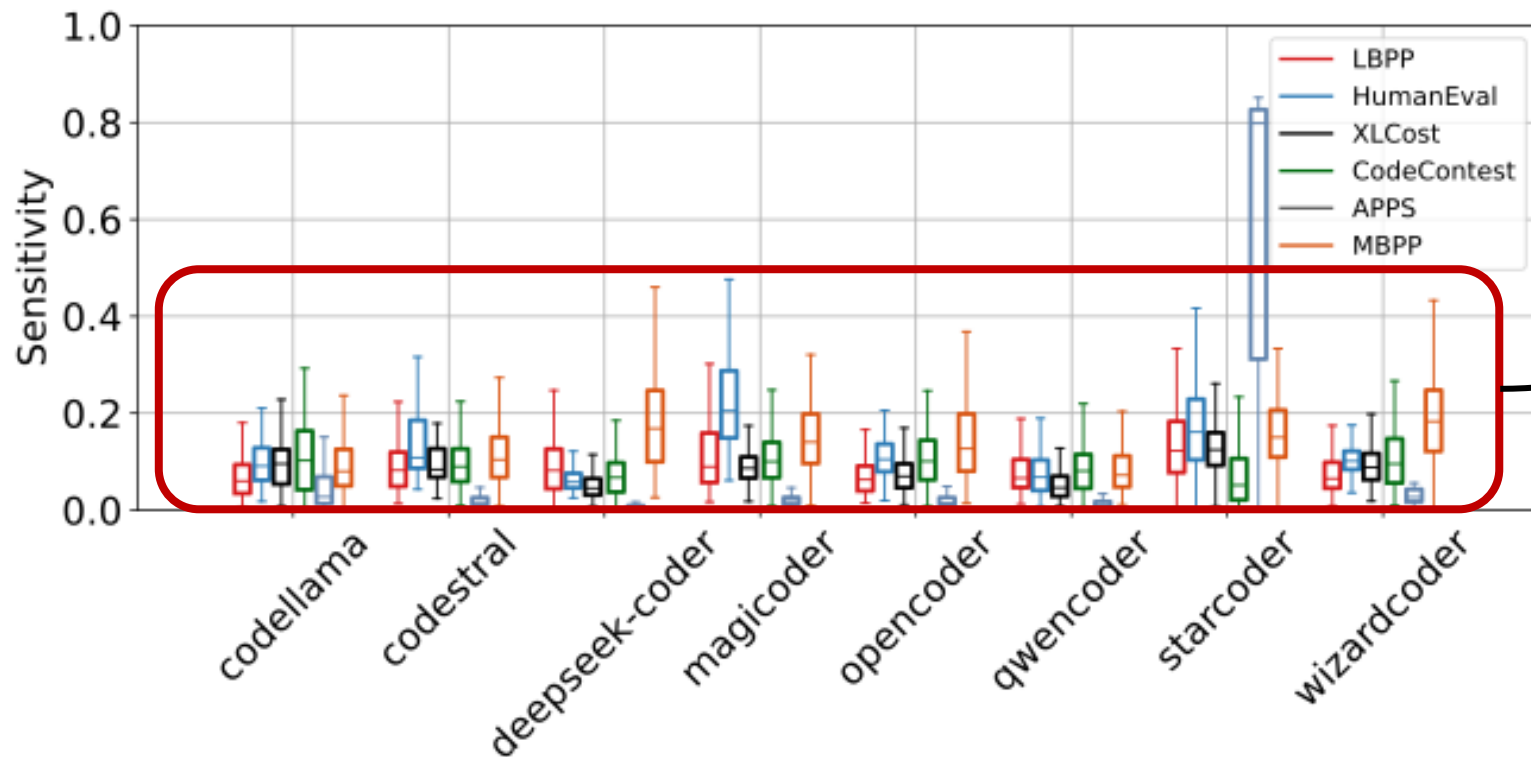
```
def is_valid_parenthesization(_IzAQImhU):
    depth = 0
    for paren in _IzAQImhU:
        if paren == '(':
            depth += 1
        else:
            depth -= 1
            if depth < 0:
                return False
    return True
```

HEAVY PERTURBATION

```
def is_valid_parenthesization(_MccOxIiI):
    _axMKjmDA = 0
    for _LeylfcSP in _MccOxIiI:
        if _LeylfcSP == '(':
            _axMKjmDA += 1
        else:
            _axMKjmDA -= 1
            if _axMKjmDA < 0:
                return False
    return True
```

Toward PEARL's Application?

Code Synthesis

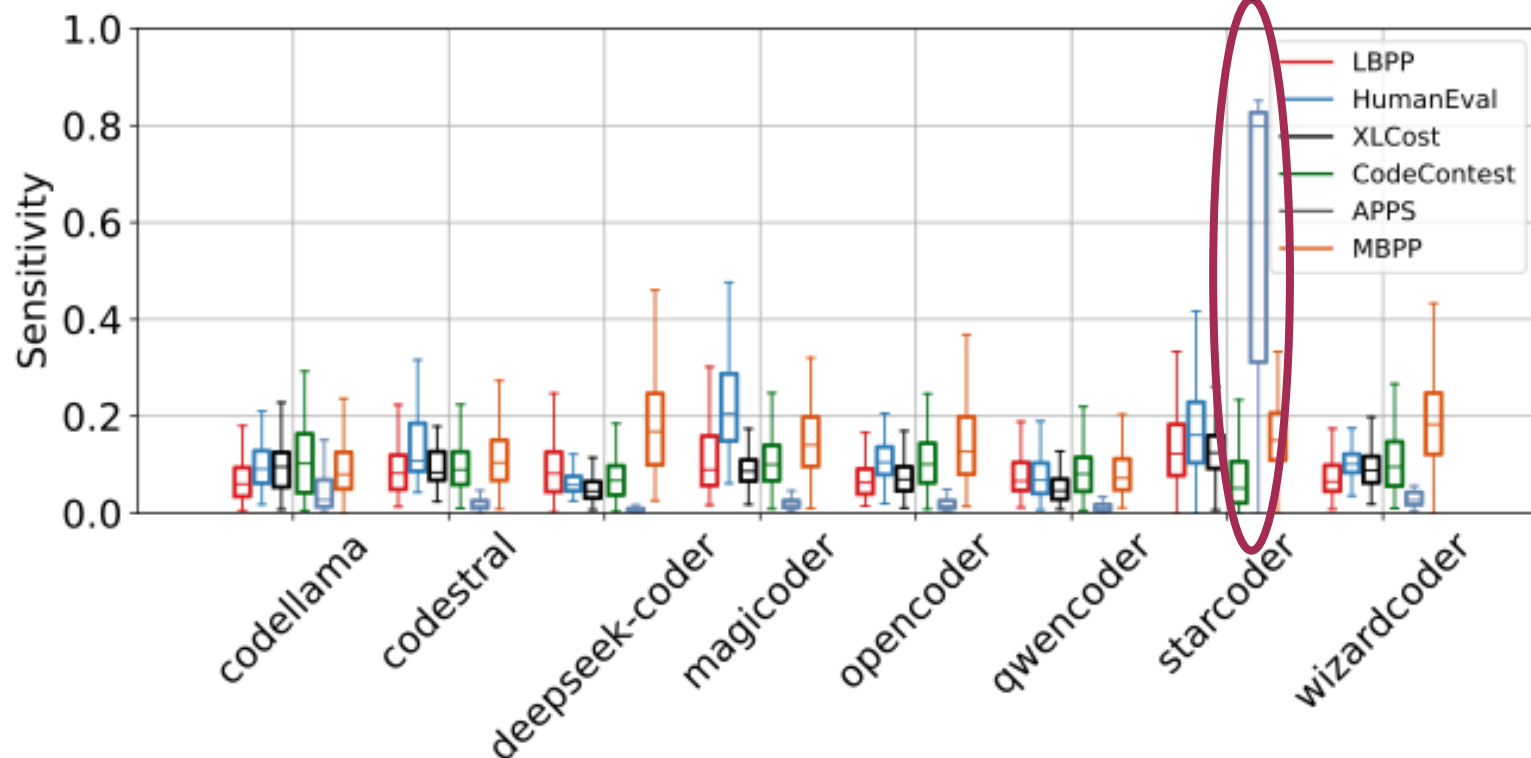


Finding - 1

*Low sensitivity across most models :
Strong generalization capabilities for
basic code generation task*

Toward PEARL's Application?

Code Synthesis

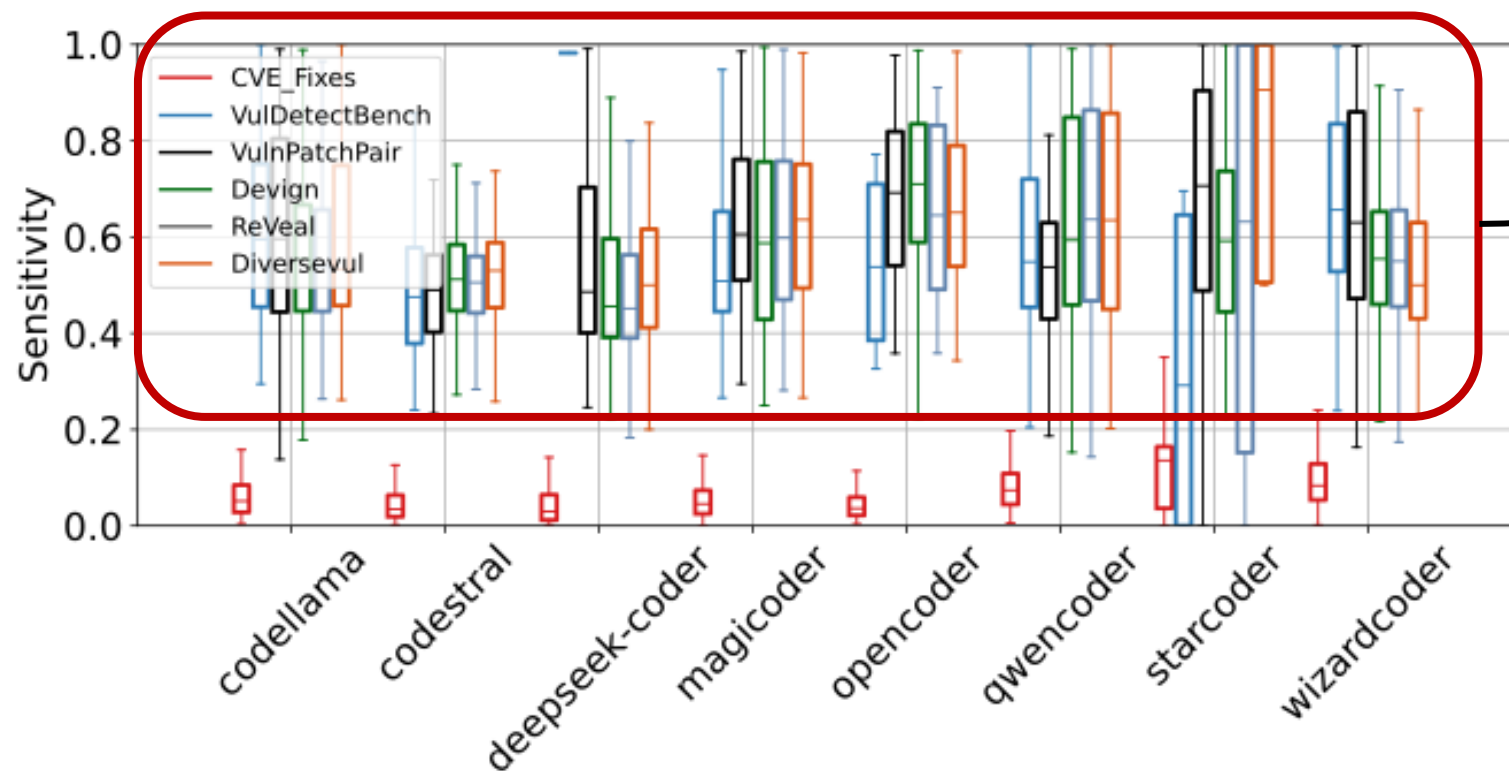


Finding - 2

High sensitivity of StarCoder on APPS benchmark : potentially memorized.

Toward PEARL's Application?

Vulnerability Detection

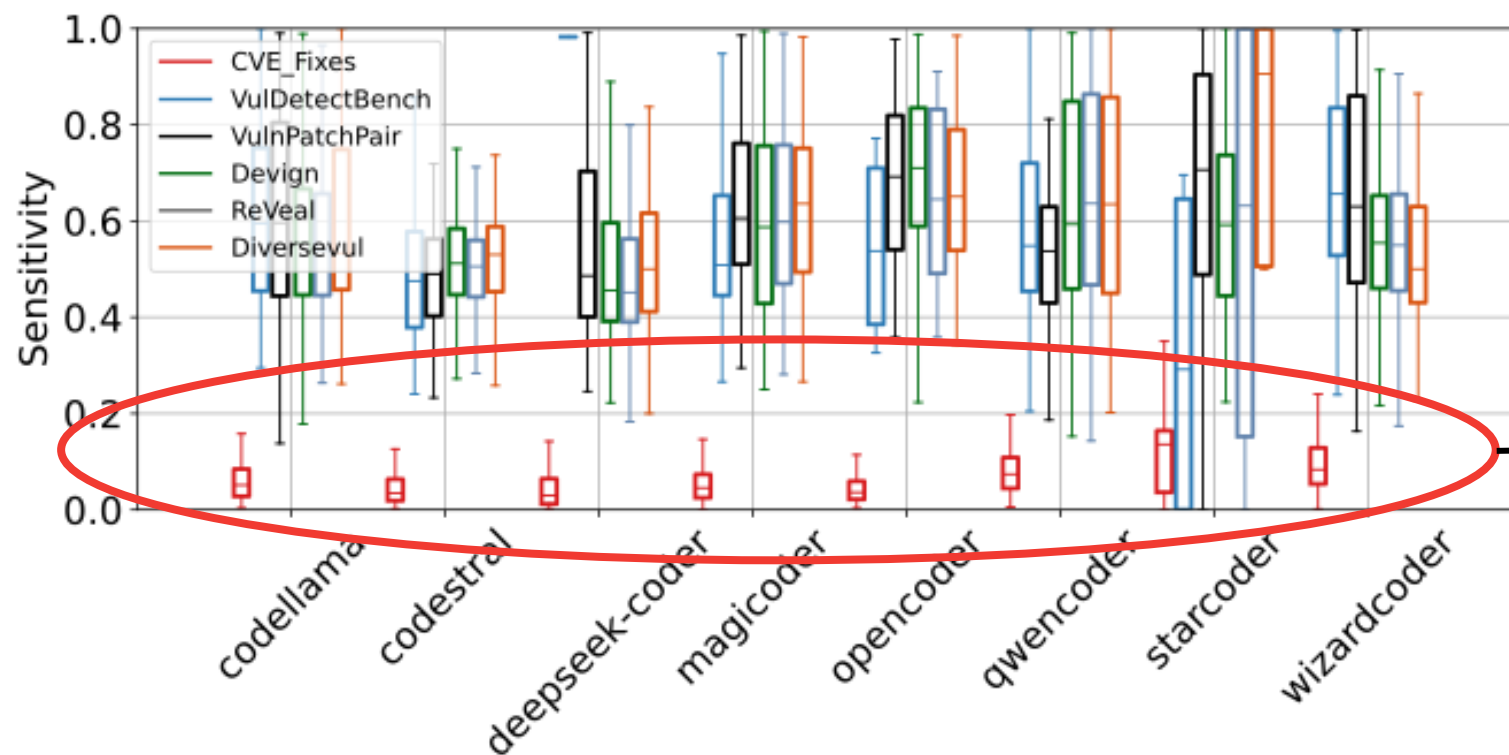


Finding - 3

*High sensitivity of other benchmarks :
generalisation challenges for security
tasks following the complexity*

Toward PEARL's Application?

Vulnerability Detection

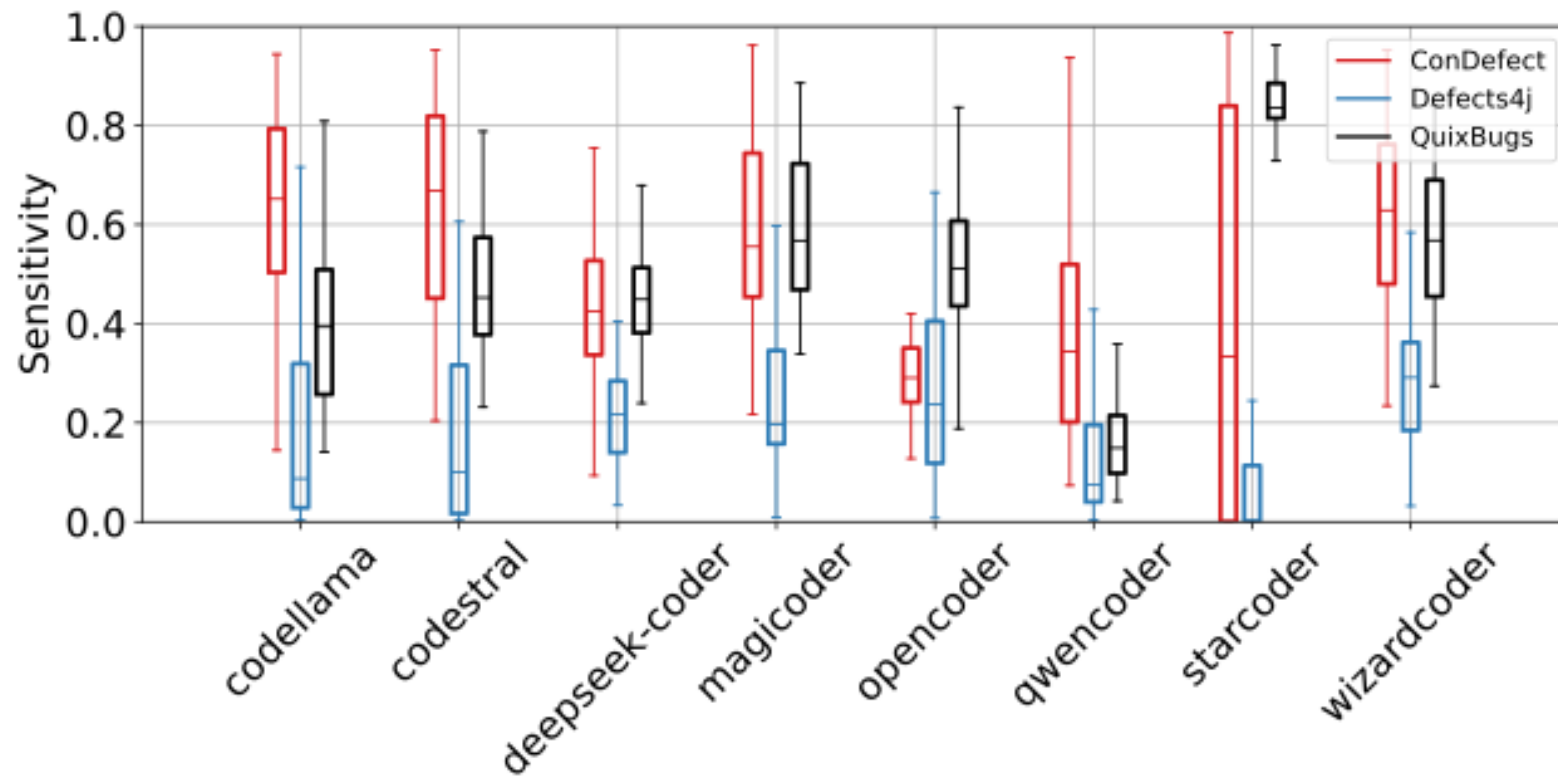


Finding - 4

*Low sensitivity of CVE_fixes :
generalized understanding of
common vulnerability pattern*

Toward PEARL's Application?

Program Repair

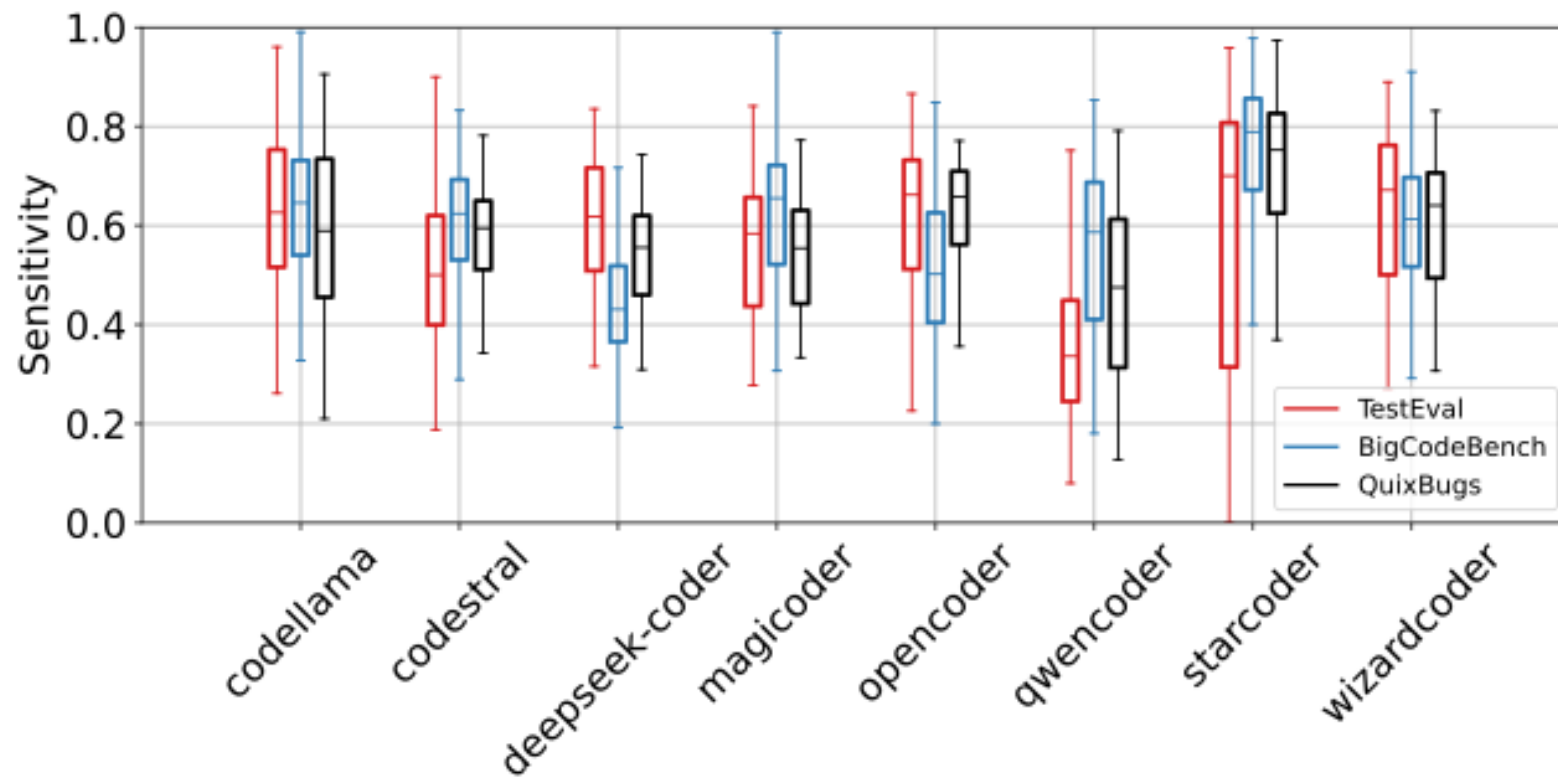


Finding - 5

Low sensitivity of Defects4J:
*strong generalization capabilities
for type of bugs related.*

Toward PEARL's Application?

Test Generation

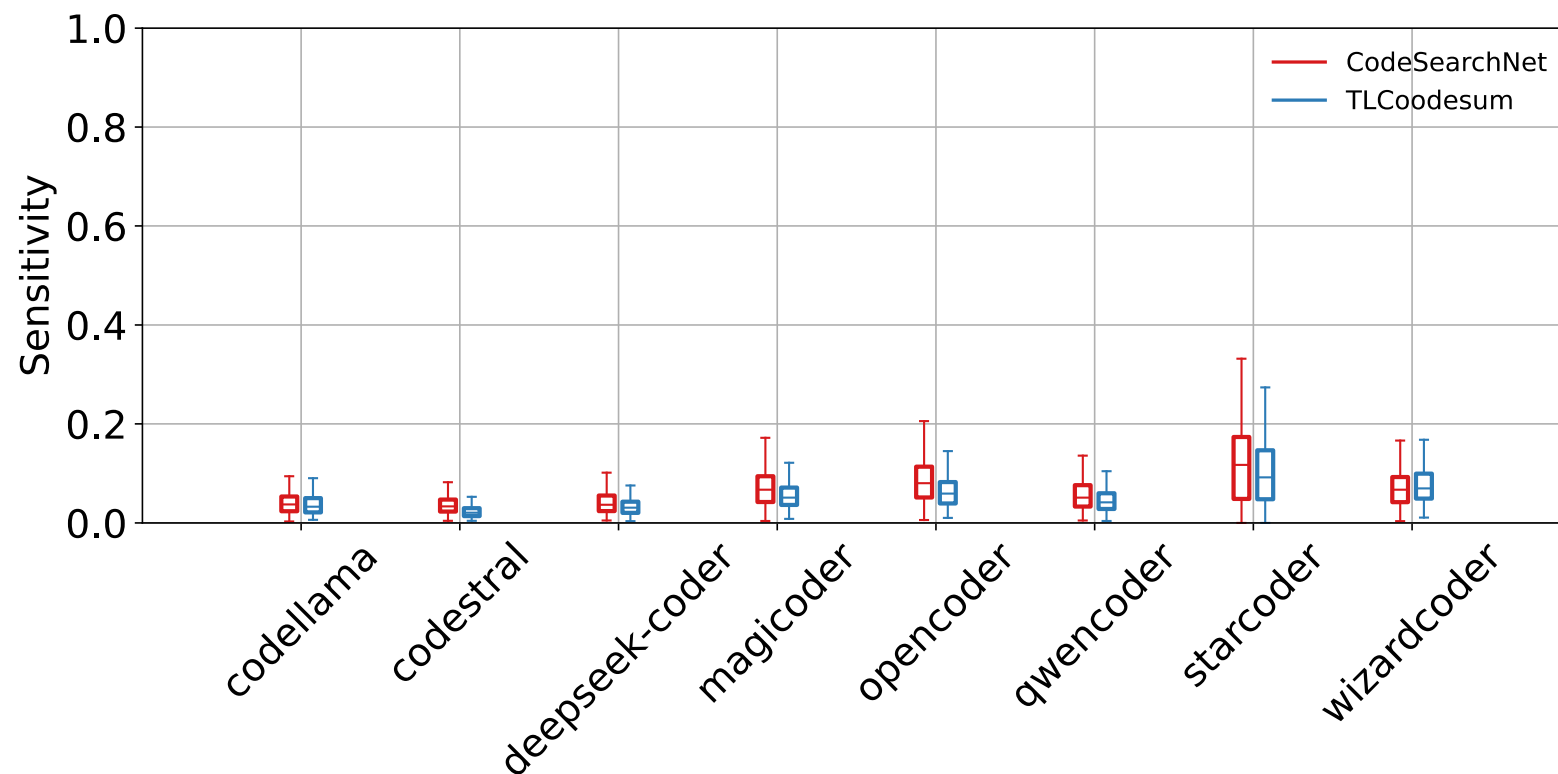


Finding – 6

High sensitivity across all models :
more challenging generalization task for current code LLMs

Toward PEARL's Application?

Code Summarization



Finding - 7

Low sensitivity of across code

**summarization benchmarks : strong
generalization capabilities for this task.**

To sum up



Task-Dependent Generalization Capabilities :

Some tasks remain challenging for LLM in term of generalization (Program Repair – Test Generation)



High performance != memorization :

High performance demonstrated by a model does not imply necessary memorization.



Model specific pattern :

Architecture and training methodology matter far more than scale.

Thank you for you attention !

All questions are welcome.

Memorization or interpolation? detecting llm memorization through **input perturbation analysis**

[PDF] [arxiv.org](#)

[AE Djiré](#), [AK Kaboré](#), [ET Barr](#), [J Klein](#)... - arXiv preprint arXiv ..., 2025 - [arxiv.org](#)

... 's performance is to **input perturbations**, enabling memorization detection without requiring access to the model's internals. We investigate how **input perturbations** affect the consistency ...

☆ Enregistrer Citer Cité 4 fois Autres articles »

Learned or Memorized? Quantifying Memorization Advantage in Code LLMs

[PDF] [arxiv.org](#)

[DA Euraste](#), [KA Kader](#), [J Samhi](#), [ET Barr](#), [J Klein](#)... - arXiv preprint arXiv ..., 2026 - [arxiv.org](#)

The lack of transparency about code datasets used to train large language models (LLMs) makes it difficult to detect, evaluate, and mitigate data leakage. We present a perturbation-based method to quantify memorization advantage in code LLMs, defined as the performance gap between likely seen and unseen inputs. We evaluate 8 open-source code LLMs on 19 benchmarks across four task families: code generation, code understanding, vulnerability detection, and bug fixing. Sensitivity patterns vary widely across models and ...

☆ Enregistrer Citer Autres articles »